

STRINGS

What we know about Strings?

1. They are enclosed in double quotes, a type of non-primitive datatype.
2. They are multi-value data, with default value as null.
3. The length of String is not fixed – It can be zero or any positive integer.
4. It can accept alphanumeric data also.

Strings in C programming language:

Let's take an example String as "group". This data has 5 characters together. In C, String is stored as an Array of characters with null character in the end.

g	r	o	u	p	\0
---	---	---	---	---	----

To access the characters and print them, we have to use looping statements.

Strings in Java:

- ❖ But in Java, Strings are stored as Objects.
- ❖ Objects are used to store multi-valued data.
- ❖ It can store not only data but also methods that are used to manipulate that data such as convert from upper to lower case and vice-versa and so on.

Built-In classes in Java to store Strings:

1. java.lang.String
2. java.lang.StringBuilder
3. java.lang.StringBuffer

String Class:

- 'String' is a ClassName, a non-primitive datatype used to declare non-primitive variables.
- The non-static methods of the Object class such as toString(), equals(Object o), hashCode() are by default over-ridden in built-in String class.

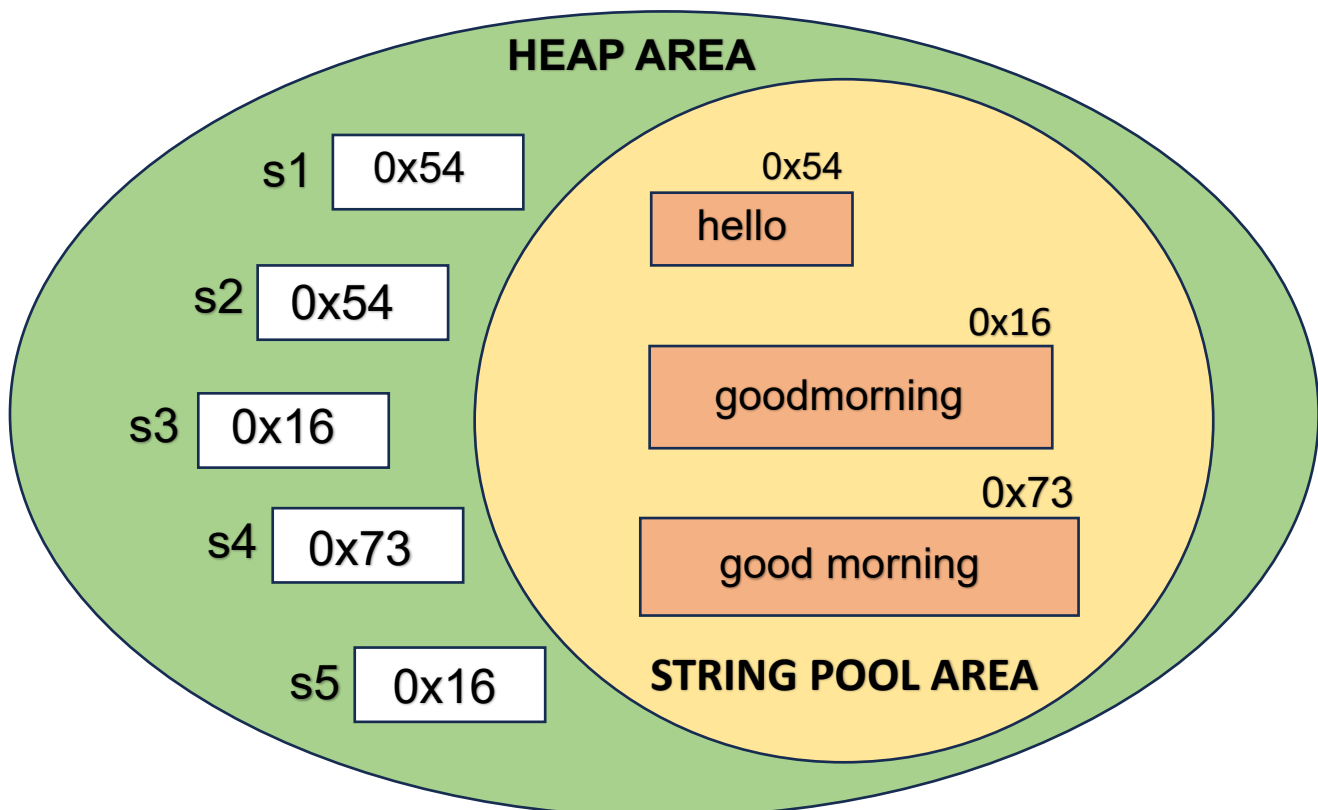
Important Constructors of String class:

1. String()
2. String(String)
3. String(char [])
4. String(StringBuilder)
5. String(StringBuffer)

Creating String object using string literal:

1. When we use a String literal in a program, an object of java.lang.String class is created in String pool area of Heap area, only if object does not exist already.
2. If already exists, the reference of the existing object is returned.

```
2                                     true
3 public class SampleDemo {         false
4 public static void main(String[] args) {   true
5     String s1 = "hello";           true
6     String s2 = "hello";
7     System.out.println(s1==s2);
8     String s3 = "goodmorning";
9     String s4 = "good morning";
10    System.out.println(s3==s4);
11    String s5 = s3;
12    System.out.println(s5=="goodmorning");
13    s1=s3;
14    System.out.println(s1=="goodmorning");
15 }
16 }
17
```



Creating String object using new keyword/operator:

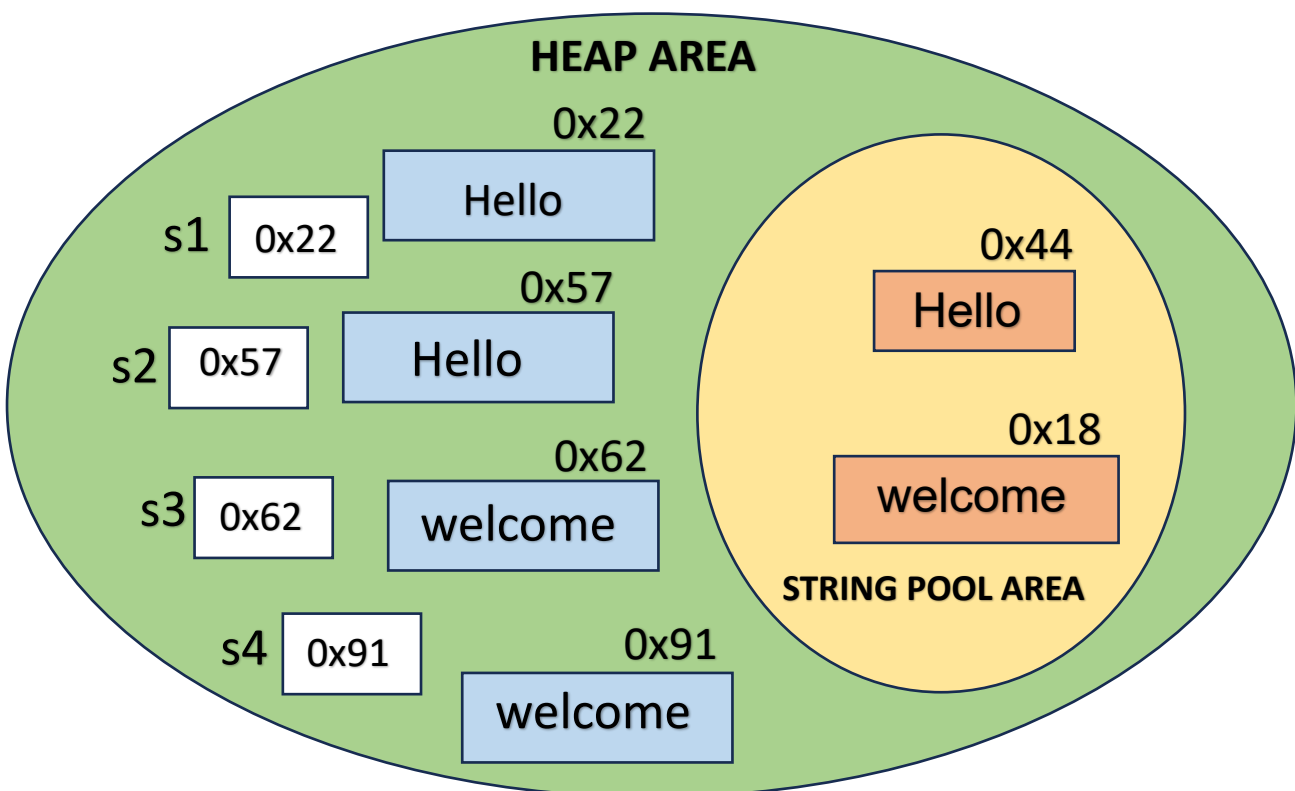
1. The syntax to create String object using new keyword is:

String variable_name = new String("string_data");

2. String object is created in Heap area and not in String pool area.
3. The methods of Object class like equals(Object o), hashCode() are not over-ridden for StringBuilder and StringBuffer classes

```
1 package collectionInterface;
2
3 public class SampleDemo {
4     public static void main(String[] args) {
5         String s1 = new String("Hello");
6         String s2 = new String("Hello");
7         System.out.println("****STRING CLASS****");
8         System.out.println(s1==s2);
9         // compares reference, which are different
10        System.out.println(s1.equals(s2));
11        // compares states, since equals(Object o)
12        // is over-ridden in String class
13        System.out.println(s1=="Hello");
14        System.out.println("****STRINGBUFFER CLASS****");
15        StringBuffer s3 = new StringBuffer("welcome");
16        StringBuffer s4 = new StringBuffer("welcome");
17        System.out.println(s3==s4);
18        // compares reference, which are different
19        System.out.println(s3.equals(s4));
20        // does not compare states, since equals(Object o)
21        // is not over-ridden in StringBuffer class
22    }
23 }
24
```

```
<terminated> SampleDemo [Java Application] C:
****STRING CLASS****
false
true
false
****STRINGBUFFER CLASS****
false
false
```



Creation of Object with respect to three classes:

1. **String class:**

- Using literals: String s1 = "hello";
- Using 'new' keyword: String s2 = new String("welcome");

2. **StringBuffer class:**

Only using 'new' keyword: StringBuffer s3 = new StringBuffer("bye");

3. **StringBuilder class:**

Only using 'new' keyword: StringBuilder s4 = new StringBuilder("hey");

Mutability of String Objects:

- ❖ String class objects are **immutable** in nature.
- ❖ StringBuilder and StringBuffer class objects are **mutable** in nature.

IMMUTABLE:

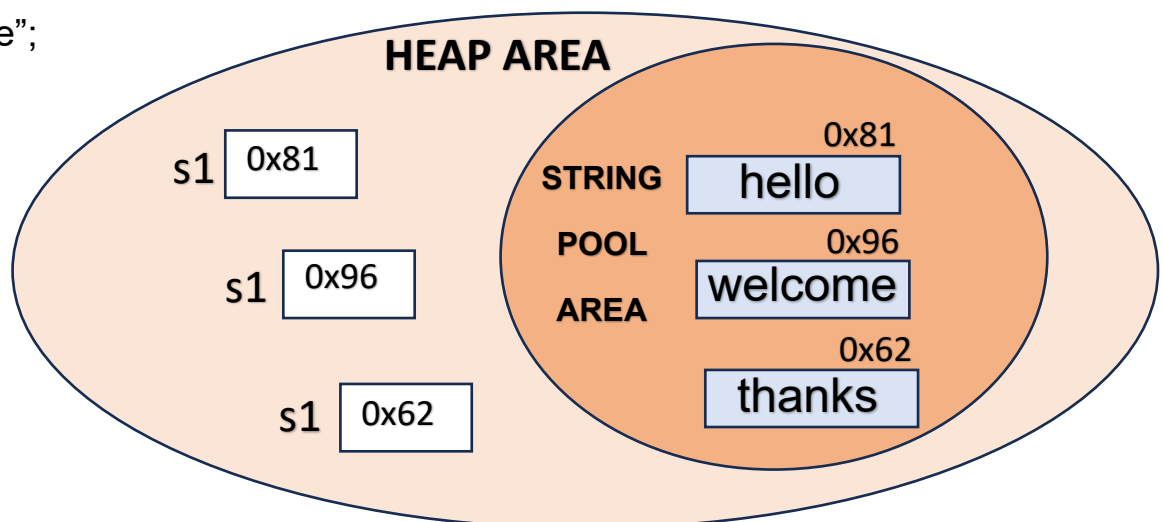
For every modification of String data, a new object is created in String pool area.

String s1 = "hello";

Internally, compiler creates object like String s1 = new String("hello");

s1 = "welcome";

s1 = "thanks";



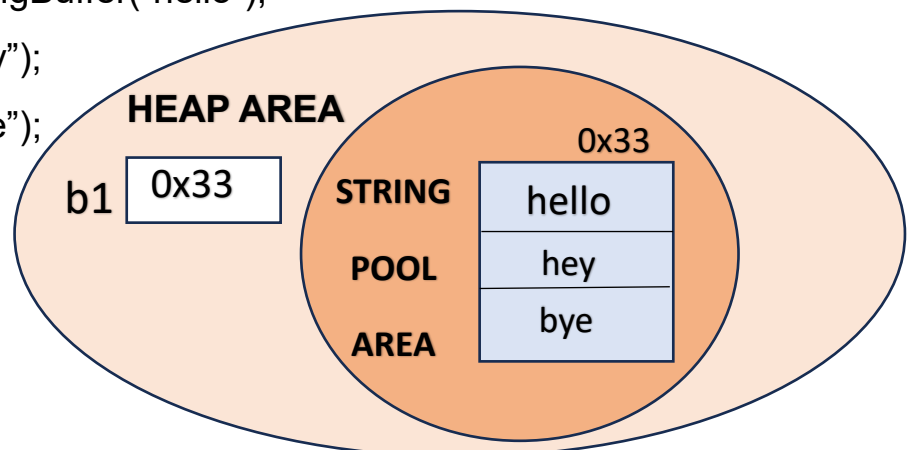
MUTABLE:

For every modification of String data of same variable name, it gets appended in the same object. It saves a lot of internal memory.

StringBuffer b1 = new StringBuffer("hello");

b1 = new StringBuffer("hey");

b1 = new StringBuffer("bye");



	STRING CLASS	STRING BUFFER CLASS	STRING BUILDER CLASS
1.	In-built class	In-built class	In-built class
2.	java.lang package	java.lang package	java.lang package
3.	Final class	Final class	Final class
4.	Constructors: String() String("String literals")	Constructors: StringBuffer() StringBuffer("String literals")	Constructors: StringBuilder() StringBuilder("String literals")
5.	Child to Object class: ➤ toString() ➤ equals(Object o) ➤ hashCode() are over-ridden	Child to Object class: ➤ toString() is over-ridden	Child to Object class: ➤ toString() is over-ridden
6.	Child to following interfaces: ➤ Comparable ➤ Serializable ➤ CharSequence	Child to following interfaces: ➤ Serializable ➤ CharSequence	Child to following interfaces: ➤ Serializable ➤ CharSequence
7.	Object can be created using 'new' keyword and directly by String literals as well	Object can be created only using 'new' keyword	Object can be created only using 'new' keyword
8.	Objects are immutable in nature	Objects are mutable in nature	Objects are mutable in nature

OBJECT CREATION:

String class - both literals and new keyword

StringBuffer class - only new keyword

StringBuilder class - only new keyword

MUTABLE:

String class is immutable

StringBuffer and StringBuilder are mutable

MULTI-THREADING:

String class is thread safe - inherently, due to immutability

StringBuffer is thread safe - due to synchronized methods(multi-threading allowed)

StringBuilder is not thread safe (multi-threading not allowed)

The key difference in multithreading is that `StringBuffer` is **thread-safe (synchronized)**, while `String` is **inherently thread-safe due to immutability**, and `StringBuilder` is **not thread-safe**. [Stack Overflow +1](#)

Here is a detailed breakdown of their differences, with a focus on multithreading:

Thread Safety and Mutability

Feature	<code>String</code>	<code>StringBuffer</code>	<code>StringBuilder</code>
Mutability	Immutable (cannot be changed after creation; modifications create new objects)	Mutable (can be modified in-place)	Mutable (can be modified in-place)
Thread-Safety	Thread-safe (due to immutability, multiple threads can access safely)	Thread-safe (all methods are <code>synchronized</code>)	Not thread-safe (methods are not synchronized)
Performance	Slow for repeated modifications (creates many objects)	Slower than <code>StringBuilder</code> due to synchronization overhead	Fastest (no synchronization overhead)
Use Case	Fixed, unchanging text (e.g., constants, messages)	String manipulation in a multi-threaded environment	String manipulation in a single-threaded environment

Multithreading Implications

- **`String`** : An immutable object can be safely shared across multiple threads without any synchronization concerns, as its value can never change after creation. This inherent thread safety makes it suitable for use in concurrent environments when the data is constant.
- **`StringBuffer`** : This class is designed specifically for multi-threaded scenarios where a mutable string needs to be accessed by multiple threads simultaneously.
 - **Synchronization**: Every method (`append()` , `insert()` , `delete()` , etc.) is marked as `synchronized` , ensuring that only one thread can execute a method on a `StringBuffer` instance at a time. This prevents data corruption and race conditions.
 - **Performance Cost**: The synchronization adds a performance overhead, making it slower than `StringBuilder` in single-threaded operations where such safety mechanisms are unnecessary.
- **`StringBuilder`** : Introduced in Java 5 as a faster alternative to `StringBuffer` for single-threaded use.
 - **No Synchronization**: Its methods are not synchronized, which improves performance by avoiding the overhead of locking and unlocking.
 - **Data Inconsistency**: If multiple threads try to modify the same `StringBuilder` object concurrently without external synchronization, data inconsistency or runtime exceptions may occur. [Unstop +6](#)

Best Practices

- If your string data is constant or rarely changes, use the `String` class.
- If you are manipulating a string frequently in an environment where multiple threads might access the same object, use `StringBuffer` to ensure thread safety.
- If you are working in a single-threaded application or a method with a local string variable (where only one thread will access it), use `StringBuilder` for maximum performance. [Medium](#)